

UNIVERSIDADE DE SÃO PAULO  
INSTITUTO DE FÍSICA DE SÃO CARLOS

NICOLAS OLIVEIRA ROSSONI

Redes neurais aplicadas na física: usando PINN e MixFunn para  
resolver EDPs

São Carlos

2026

NICOLAS OLIVEIRA ROSSONI

## Redes neurais aplicadas na física: usando PINN e MixFunn para resolver EDPs

Trabalho de Conclusão de Curso apresentado ao Instituto de Física de São Carlos da Universidade de São Paulo para obtenção do título de Bacharel em Física.

Orientador: Prof. Dr. Celso Jorge Villas-Boas — UFS-Car

São Carlos

2026

## RESUMO

Recentes avanços em *hardware* e bibliotecas de aprendizado profundo — em particular a diferenciação automática como ferramenta nativa — abriram caminho para representar equações diferenciais parciais (EDPs) de forma compacta e geral por meio de redes neurais. Este trabalho explora duas arquiteturas dessa linha, *Physics-Informed Neural Networks* (PINN) e *Mix2Funn* (MixFunn), aplicando-as a problemas físicos diversos para validar os elementos centrais do paradigma. Primeiramente, o escoamento de Kovasznay — caso particular das equações de Navier–Stokes com solução analítica fechada — foi usado como base para um estudo aprofundado: variação de arquitetura, comparação entre os regimes supervisionado e não-supervisionado, análise do decaimento da *loss*, métodos de regularização (*dropout* e subamostragem), e técnicas específicas da MixFunn (*pruning* de funções atômicas e *annealing* da temperatura da *softmax*). Em seguida, ambas as redes foram aplicadas ao escoamento sanguíneo em um aneurisma intracraniano tridimensional, usando dados de paciente real do *dataset* ANEUMO, comparando os regimes supervisionado e não-supervisionado em um problema de geometria complexa. Por fim, três casos complementares (Burgers viscosa, Schrödinger não-linear, e magnetostática 2D com interface dielétrica) foram apresentados como demonstração condensada da diversidade de EDPs cobertas. Conclui-se que ambas as redes são ferramentas viáveis para resolução de EDPs, com perfis complementares: a PINN oferece capacidade expressiva universal, e a MixFunn traz interpretabilidade e simplicidade. Cada problema exige exploração própria de seu espaço de soluções e hiperparâmetros, semelhante mais ao trabalho de um laboratório empírico do que à execução determinística de um método numérico tradicional.

**Palavras-chave:** EDP. PINN. MixFunn. Deep Learning. Navier–Stokes.

## 1 INTRODUÇÃO

O objetivo deste trabalho se divide em duas frentes. A primeira é exploratória: estudar como redes neurais podem ser empregadas para resolver equações diferenciais parciais (EDPs) em física, entender como funcionam, em que regimes convergem, como falham quando falham, e o que cada arquitetura revela sobre a estrutura do problema. Sobre essa base, a segunda frente é comparativa: avaliar duas arquiteturas específicas, a *Physics-Informed Neural Network* (PINN) [Raissi 2019], e a *Mix2Funn* [Farias 2025], em uma sequência de problemas que vão da mecânica clássica à mecânica quântica, medindo qualidade da solução e custo em relação aos métodos numéricos tradicionais, como o de Runge-Kutta.

### 1.1 O que é uma rede neural (MLP)

As redes neurais, em geral, são algoritmos que aplicam uma transformação em um conjunto de dados de entrada (*input*), podendo variar a sua dimensionalidade na saída (*output*). Elas têm a liberdade para

manipular a entrada com funções lineares e não lineares, correlacionando cada informação entre si.

Elas possuem várias camadas e diversos parâmetros que modificam essa transformação. Dependendo da estrutura da rede e dos parâmetros escolhidos, ela se torna um “aproximador universal” de qualquer transformação desejada. Quanto maior a liberdade da rede para modificar os dados (mais parâmetros e camadas), maior sua expressividade.

Para uma dada “transformação ideal”, existe um conjunto de parâmetros capaz de aproximar esse comportamento, para um dado erro, se a rede for expressiva o suficiente. A partir disso, podemos usar métodos de otimização, como o *backpropagation* combinado com gradiente descendente, para encontrar esses parâmetros, de forma supervisionada (com dados reais de um sistema) ou não supervisionada (sem os dados).

A rede neural mais comum é a MLP (*multilayer perceptron*), uma rede totalmente conectada que organiza essa transformação em camadas sucessivas. Cada neurônio de uma camada multiplica a entrada por seus próprios pesos, soma um viés e aplica uma função de ativação não-linear; a saída de uma camada é a entrada da seguinte, como mostra o diagrama da Figura 1.

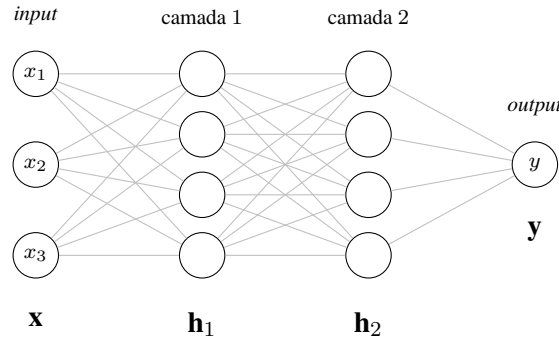


Figura 1: Diagrama de uma MLP de duas camadas ocultas. Cada aresta carrega um peso  $W_{ji}^{(l)}$ ; cada nó das camadas ocultas soma um viés  $b_j^{(l)}$  e aplica  $\sigma$ .

Componente a componente, a saída do  $j$ -ésimo neurônio da camada  $l + 1$  é

$$h_j^{(l+1)} = \sigma \left( \sum_{i=1}^{n_l} W_{ji}^{(l)} h_i^{(l)} + b_j^{(l)} \right), \quad l = 0, 1, \dots, L - 1, \quad (1)$$

com  $h_i^{(0)} = x_i$  e saída final  $y_k = \sum_j W_{kj}^{(L)} h_j^{(L)} + b_k^{(L)}$ . Empilhando os pesos  $W_{ji}^{(l)}$  em uma matriz  $W_l$  e os vieses  $b_j^{(l)}$  em um vetor  $\mathbf{b}_l$ , a equação (1) fica na sua forma vetorial:

$$\mathbf{h}_0 = \mathbf{x}, \quad \mathbf{h}_{l+1} = \sigma(W_l \mathbf{h}_l + \mathbf{b}_l), \quad \mathbf{y} = W_L \mathbf{h}_L + \mathbf{b}_L. \quad (2)$$

A escolha da função de ativação  $\sigma$  é livre, e há diversas opções na prática. Em problemas da física, a tangente hiperbólica  $\sigma(z) = \tanh(z)$  é a mais comum, por ser suave e ter derivadas bem comportadas em qualquer ordem, exatamente o que será requisitado mais adiante. Em problemas de

outra natureza, como reconhecimento de imagem, em que a suavidade da saída em relação à entrada não é importante, usa-se outra família. Um exemplo é a *rectified linear unit* (ReLU), que vale zero para argumento negativo e é a identidade para argumento positivo,  $\sigma(z) = \max(0, z)$ .

Definida a arquitetura, treinar a rede vira um problema de busca: dado um número grande de parâmetros, queremos encontrar o conjunto que melhor aproxima a transformação-alvo. O método canônico parte da função de perda (“*loss*”), que é a maneira de quantificar o erro da rede. No caso supervisionado mais comum, em que temos  $N$  pontos  $(\mathbf{x}_i, \mathbf{y}_i)$  e queremos que a rede  $u_\theta$  os reproduza, a perda é o erro quadrático médio:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|u_\theta(\mathbf{x}_i) - \mathbf{y}_i\|^2. \quad (3)$$

Como a perda é uma função explícita de cada parâmetro  $\theta_j \in \{W_l, \mathbf{b}_l\}$ , conseguimos calcular o gradiente  $\partial \mathcal{L} / \partial \theta_j$ , que mede quanto a perda muda quando aquele parâmetro varia. O *backpropagation* é o algoritmo que aplica a regra da cadeia ao longo das camadas para obter esse gradiente de forma eficiente. De posse dele, damos um passo pequeno (de tamanho controlado pela “*learning rate*”  $\eta$ ) na direção contrária ao gradiente:

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial \mathcal{L}}{\partial \theta_j}. \quad (4)$$

Iterando esse passo, a perda decresce e os parâmetros se aproximam de um conjunto que aproxima a transformação desejada.

Com isso fica descrito o funcionamento básico das MLPs. Existem outras arquiteturas em uso na ciência de dados moderna, convolucionais, recorrentes, *transformers*, entre outras —, mas a que nos interessa aqui é uma reformulação específica da MLP para resolver EDPs: a PINN.

## 1.2 Como redes neurais resolvem EDPs (PINN)

Neste trabalho, a “transformação ideal” a ser aproximada não vem de dados experimentais, mas de uma EDP, de modo que a rede aprenda a solução. O objetivo é obter soluções de forma mais eficiente do que os métodos numéricos tradicionais, como o de Runge-Kutta.

Para entender por que isso é viável, vale um parêntese histórico. A ascensão da inteligência artificial transformou o cálculo de gradientes em *commodity*: GPUs com processadores especializados e bibliotecas robustas, hoje em larga escala, são produtos voltados precisamente a computar derivadas de funções compostas com máxima eficiência. Em qualquer biblioteca moderna de *deep learning*, todo o grafo de derivadas da rede é construído junto com o *forward pass*, a operação inversa é apenas uma multiplicação recursiva ao longo desse grafo. Obter  $\partial u / \partial x$  ou  $\partial^2 u / \partial t^2$  a partir de uma rede  $u_\theta(x, t)$  é, literalmente, uma linha de código.

Esse fato é o que viabiliza usar redes neurais como solução de EDPs. Suponha que a rede  $u_\theta(x, t)$  recebe o ponto  $(x, t)$  como entrada e devolve o valor da função  $u$  como saída, conforme o diagrama

da Figura 2.

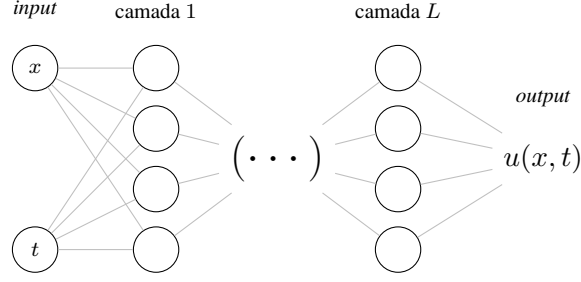


Figura 2: Rede neural que representa a solução de uma EDP como uma função  $u(x, t)$ : as entradas  $x$  e  $t$  alimentam  $L$  camadas ocultas e a saída devolve o valor da função no ponto. A profundidade da rede é livre, indicada por  $(\dots)$ .

Por diferenciação automática, obtemos  $\partial u / \partial x$ ,  $\partial u / \partial t$ ,  $\partial^2 u / \partial x^2$  e assim por diante, em qualquer ordem, em qualquer ponto do domínio, sem precisar de uma malha discreta. A partir disso, fica claro que conseguimos construir a função de perda em função das próprias derivadas: se a EDP é  $\mathcal{F}[u] = 0$  no interior do domínio, com condição inicial  $u(x, 0) = u_0(x)$  e condição de contorno  $u(x, t) = g(x, t)$  na borda, definimos

$$\mathcal{L}(\theta) = \underbrace{\sum_{i=1}^{N_r} \frac{|\mathcal{F}[u_\theta](x_i, t_i)|^2}{N_r}}_{\mathcal{L}_{\text{res}}} + \underbrace{\sum_{i=1}^{N_{\text{CI}}} \frac{|u_\theta(x_i, 0) - u_0(x_i)|^2}{N_{\text{CI}}}}_{\mathcal{L}_{\text{CI}}} + \underbrace{\sum_{i=1}^{N_{\text{CC}}} \frac{|u_\theta(x_i, t_i) - g(x_i, t_i)|^2}{N_{\text{CC}}}}_{\mathcal{L}_{\text{CC}}}, \quad (5)$$

onde os pontos  $(x_i, t_i)$  são amostrados, respectivamente, no interior do domínio (resíduo), na condição inicial e na condição de contorno. Minimizando  $\mathcal{L}$  pelo procedimento da seção anterior, a rede converge para uma função que satisfaz simultaneamente a EDP e as condições.

A beleza dessa formulação é que o treino é não-supervisionado: não precisamos de uma solução numérica de referência nem de medições experimentais. A rede aprende a partir do próprio enunciado do problema, a EDP e suas condições inicial e de contorno são toda a informação necessária. Essa é a essência da *Physics-Informed Neural Network* (PINN), introduzida em [Raissi 2019].

Uma vez treinada, a rede  $u_\theta$  é uma representação contínua da solução em todo o domínio: cada avaliação requer uma passagem direta pelo grafo, da ordem de milissegundos. Métodos numéricos clássicos precisam discretizar o domínio em uma malha de antemão, e o custo escala linearmente com o número de pontos da grade — da ordem de segundos, várias ordens de magnitude acima da consulta à rede, para os problemas tratados neste trabalho. A discrepância piora em domínios bidimensionais e tridimensionais, em que o número de pontos cresce com a potência da dimensão. A PINN troca um custo alto de treino, único, por consultas baratas e pontuais durante o uso.

### 1.3 O que são MixFunnns

A MLP da PINN é uma arquitetura “cega”: como suas ativações são funções genéricas (tipicamente  $\tanh$ ), a rede precisa redescobrir, camada a camada, funções comuns em soluções da física (senos, exponenciais, polinômios). Isso pesa duas vezes: muitos parâmetros para uma estrutura simples, e a função final fica espalhada em milhares de pesos sem significado individual. A *Mix2Funn* [Farias 2025] reformula o neurônio com dois ganhos em mente. Primeiro, “viés indutivo”: em vez de redescobrir funções, a rede recebe um conjunto fixo de funções elementares como conhecimento prévio do tipo de solução esperada, o que reduz drasticamente o número de parâmetros necessário para representar a solução. Segundo, “interpretabilidade”: como cada peso aparece multiplicando uma função simbólica, após o treino podemos ler a expressão analítica da solução diretamente dos pesos remanescentes) a rede entrega uma fórmula, não só uma tabela de valores.

A construção do neurônio se dá em dois estágios. Dado um vetor de entrada  $\mathbf{x} = (x_1, \dots, x_n)$ , o primeiro estágio é uma *pré-mistura quadrática*: além das  $n$  entradas, formamos também todos os  $n(n-1)/2$  produtos de pares cruzados  $x_i x_j$ , com  $i < j$  (sem repetir índices). Esses  $n + n(n-1)/2$  termos são os candidatos a entrar nas funções da camada seguinte.

No segundo estágio, fixamos um conjunto de  $K$  funções elementares  $\{f_k\}_{k=1}^K$ , adotamos

$$\{f_k\} = \{\sin z, \cos z, e^z, \log |z|, z, z^2, 1\}.$$

Para cada função  $f_k$ , calculamos um argumento afim  $z_k$  a partir dos termos do estágio anterior, e somamos as funções com pesos não-negativos  $\alpha_k$ . Componente a componente,

$$z_k = \sum_{i=1}^n w_i^{(k)} x_i + \sum_{i<j} w_{ij}^{(k)} x_i x_j + b_k, \quad \text{Mix2Funn}(\mathbf{x}) = \sum_{k=1}^K \alpha_k f_k(z_k). \quad (6)$$

Comparando (6) com a forma escalar da MLP (1), a única diferença adicional é o somatório externo em  $k$ , que combina respostas de várias funções de ativação em vez de uma só. A pré-mistura quadrática aparece como o termo  $\sum_{i<j} w_{ij}^{(k)} x_i x_j$ , ausente em (1).

Vale notar que a Figura 3 representa um *único* neurônio, na arquitetura, ele substitui o neurônio tradicional da MLP descrito antes. O custo em parâmetros, porém, não é o mesmo: para  $n = 2$  entradas, um neurônio MLP tem  $n + 1 = 3$  parâmetros, enquanto um neurônio Mix2Funn tem  $K(n + n(n-1)/2 + 1) + K = 7 \cdot 4 + 7 = 35$  parâmetros. Cada neurônio Mix2Funn vale, em parâmetros, cerca de uma dúzia de neurônios da MLP, mas já vem com a base de funções pré-instalada.

Os pesos  $\alpha_k$  que ponderam as funções precisam ser positivos: caso contrário, contribuições com sinais opostos podem se cancelar dentro do mesmo neurônio, o que dificultaria identificar quais funções de fato participam da solução. Garantimos essa positividade pela *softmax*, que recebe um vetor

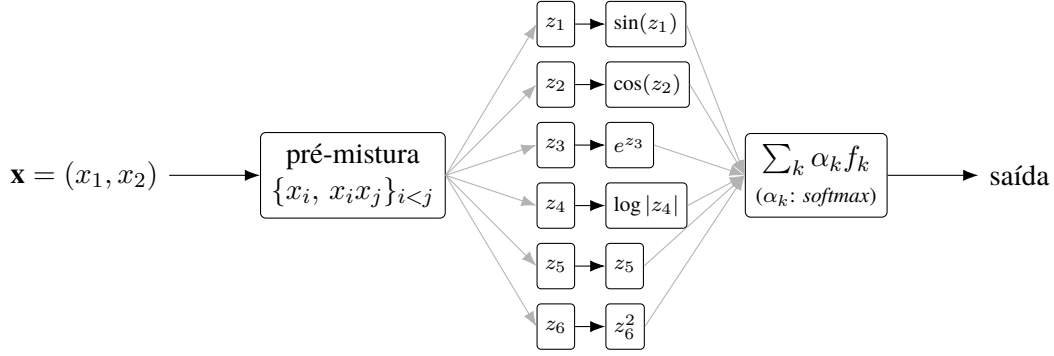


Figura 3: Neurônio Mix2Funn para  $\mathbf{x} = (x_1, x_2)$ . À esquerda, a pré-mistura forma  $\{x_1, x_2, x_1x_2\}$  a partir das entradas. À direita, cada uma das  $K$  funções da base recebe seu próprio argumento afim  $z_k$ , e as saídas são combinadas com pesos  $\alpha_k$  não-negativos (*softmax*). Para evitar saturação visual, mostramos 6 das 7 funções (a constante 1 é omitida).

de pesos brutos  $\beta = (\beta_1, \dots, \beta_K)$ , ajustados livremente pelo otimizador, e devolve

$$\alpha_k = \frac{e^{\beta_k/T}}{\sum_{j=1}^K e^{\beta_j/T}}, \quad \alpha_k \geq 0, \quad \sum_k \alpha_k = 1. \quad (7)$$

A *softmax* é, em essência, uma normalização exponencial: ela transforma qualquer vetor real em uma distribuição de probabilidade. O parâmetro  $T$ , chamado “temperatura”, controla o quão concentrada é essa distribuição:  $T$  grande achata o vetor em uma distribuição quase uniforme entre as  $K$  funções (todas contribuem parecido), enquanto  $T$  pequeno acentua o maior peso e suprime os demais (uma função domina).

Camadas empilhadas usam uma matriz de combinação triangular estrita, em que cada neurônio da camada seguinte combina apenas neurônios anteriores. Essa restrição é o que permite, em duas camadas, gerar produtos de pares de funções da base, por exemplo,  $\sin(\pi x) e^{-\alpha\pi^2 t}$ , solução típica da equação do calor.

A interpretabilidade prometida no início desta seção depende de chegarmos a uma solução em que poucas funções tenham peso significativo. Duas técnicas, usadas em conjunto durante o treino, conduzem a rede a esse regime. A primeira é o *annealing* da temperatura: começamos com  $T$  alto, para que a rede explore livremente todas as funções com pesos parecidos, e diminuimos  $T$  gradualmente ao longo das épocas, o que afia a *softmax* e força a rede a comprometer-se com uma combinação esparsa. A segunda é o *pruning* por magnitude: ao final do treino, zeramos os pesos cuja magnitude esteja abaixo de um limiar e ajustamos a rede por alguns passos adicionais para que os pesos remanescentes absorvam o que foi descartado. As funções que sobrevivem ao *pruning*, ponderadas pelos pesos finais, formam a expressão analítica da solução.



## 1.4 Generalização

Como qualquer rede neural, a PINN e a MixFunn não bastam ajustar bem a loss durante o treino: elas também precisam *generalizar*, ou seja, aprender o padrão dos dados em vez de só decorar valores. Uma rede que generaliza bem responde de modo razoável a pontos que não viu durante o treino. Quando isso falha temos o *overfit*: a loss de treino continua descendo, mas em pontos novos a rede passa a errar, sinal de que a rede ajustou ruído em vez de padrão.

Para identificar overfit a tempo, os dados disponíveis são divididos aleatoriamente em três conjuntos. O conjunto de *treino* é o que entra no loop de otimização: o gradiente da loss é calculado nele e os pesos são atualizados. O conjunto de *validação* fica de fora desse loop e serve para ajustar hiperparâmetros (arquitetura, *learning rate*, pesos  $\lambda$  na loss); ele indica se uma escolha de hiperparâmetros generaliza ou só aprende o treino. O conjunto de *teste* é uma camada extra de proteção, ele entra só na avaliação final, para o caso de overfitarmos o conjunto de validação durante a escolha de hiperparâmetros. A métrica reportada em todo este trabalho é a  $L^2$  no conjunto de teste.

Duas técnicas comuns para melhorar a generalização são o *dropout* e a *sub-amostragem*. O *dropout* desliga aleatoriamente uma fração dos neurônios em cada passo do otimizador, forçando a rede a não depender de poucos caminhos. A *sub-amostragem* usa apenas uma fração dos pontos de colocação a cada iteração, introduzindo ruído estocástico que dificulta a memorização. Em ambos os casos o objetivo é o mesmo: impedir que a rede se especialize em particularidades do treino que não representam o padrão geral. Outros métodos de regularização e variantes destes são apresentados em detalhe no capítulo 7 de [Goodfellow 2016].

## 2 MATERIAIS E MÉTODOS

Recentemente, com a ascensão da inteligência artificial, os métodos computacionais para redes neurais passaram a ter grande demanda no mercado. Isso resultou em expressivo desenvolvimento de *hardware* e *software* para executar esses algoritmos com máxima eficiência, o que abriu novas possibilidades para a pesquisa na área. Atualmente, existem GPUs com processadores especializados e bibliotecas robustas que facilitam a implementação nessa infraestrutura, como a plataforma CUDA e a biblioteca Python PyTorch.

Para executar os experimentos deste trabalho, usamos esses recursos diretamente na nuvem: cada treino roda em um *container* efêmero com GPU NVIDIA T4, alocado sob demanda na plataforma Modal<sup>1</sup>. O custo é da ordem de centavos de dólar por minuto de GPU, e o consumo total de cada experimento cabe folgadoamente no *tier* gratuito oferecido pela plataforma. O código é editado localmente, despachado para o *container* sob demanda, e os artefatos (pesos treinados, métricas, *logs*) ficam num volume persistente que a máquina local lê para gerar figuras e análises.

Todo o código está organizado em um repositório *git*<sup>2</sup>, em que cada experimento ocupa uma pasta

---

<sup>1</sup><https://modal.com>

<sup>2</sup><https://github.com/NicolasRossoni/NNinPhysics>

com quatro *scripts* numerados: pré-processamento, treino, validação e visualização. Os hiperparâmetros são fixados no próprio *script*, sem *flags* de linha de comando, de modo que cada execução fica integralmente registrada no histórico do controle de versão.

A confiabilidade e reprodutibilidade dos resultados se apoiam em três decisões. Primeiro, métodos com componente aleatória (inicialização dos pesos, sorteio dos pontos de colocação, ordem do *batch*) recebem semente fixa em PyTorch e NumPy no início de cada execução; de posse do código e da semente, qualquer corrida reportada neste trabalho é reproduzível bit a bit. Segundo, todo conjunto de hiperparâmetros, métricas finais e tempo de parede é gravado em um arquivo `metadata.json` ao lado do modelo treinado, ao final do treino. Terceiro, a matemática de cada método (PINN e Mix2Funn) segue exatamente a definida na Introdução; quaisquer técnicas adicionais investigadas aparecem explicitamente nos Resultados como objeto de estudo, não como configuração de *baseline*.

### 3 RESULTADOS

Este TCC consolida cinco problemas de EDPs sob a filosofia de *reprodução científica fiel*: para cada problema, um *paper canon* fornece o *setup* (equação, BC, regime) e a referência numérica; PINN e MixFunn são avaliadas como redes de aproximação independentes do problema. A Seção 3.1 apresenta o *deep dive* no escoamento de Kovasznay (Navier–Stokes 2D), que serve de motivação quantitativa e introdutória à abordagem; a Seção 3.2 demonstra a chave de ouro do trabalho, aplicação em anatomia cerebrovascular humana real, usando o *dataset* ANEUMO (Li *et al.* 2025). A Seção 3.3 reúne uma visão geral qualitativa de três problemas adicionais em regimes físicos distintos, demonstrando a generalidade da abordagem.

#### 3.1 Navier–Stokes 2D, escoamento de Kovasznay

##### Definição do problema

O escoamento de Kovasznay é uma solução analítica fechada das equações de Navier–Stokes incompressíveis 2D estacionárias, com decaimento exponencial em  $x$  modulado por cosseno em  $y$ . Adotamos o domínio  $\Omega = [-0,5; 1] \times [-0,5; 1,5]$  e  $\text{Re} = 40$ . O sistema reúne momento em  $x$  (Eq.1), momento em  $y$  (Eq.2) e continuidade incompressível (Eq.3):

$$\begin{cases} u u_x + v u_y = -p_x + \text{Re}^{-1}(u_{xx} + u_{yy}), & \text{(Eq.1)} \\ u v_x + v v_y = -p_y + \text{Re}^{-1}(v_{xx} + v_{yy}), & \text{(Eq.2)} \\ u_x + v_y = 0. & \text{(Eq.3)} \end{cases} \quad (8)$$

Aplicando o esquema da Equação (5) ao sistema, a loss não-supervisionada é o resíduo dos três primeiros termos:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{n=1}^N \underbrace{\left( u u_x + v u_y + p_x - \frac{u_{xx} + u_{yy}}{\text{Re}} \right)^2}_{(\text{Eq.1})} + \underbrace{\left( u v_x + v v_y + p_y - \frac{v_{xx} + v_{yy}}{\text{Re}} \right)^2}_{(\text{Eq.2})} + \underbrace{(u_x + v_y)^2}_{(\text{Eq.3})}. \quad (9)$$

A condição de contorno é imposta pela arquitetura, fora da loss: a saída efetiva  $u$  usada no resíduo é

$$u = 1 - e^{\lambda x} \cos(2\pi y) + d(x, y) \tilde{u}, \quad \lambda = \frac{\text{Re}}{2} - \sqrt{\left(\frac{\text{Re}}{2}\right)^2 + 4\pi^2}, \quad (10)$$

onde  $\tilde{u}$  é a saída crua da rede,  $1 - e^{\lambda x} \cos(2\pi y)$  é a solução analítica de Kovasznay (com  $\lambda \approx -0,964$  para  $\text{Re} = 40$ ), e  $d(x, y)$  é uma função arbitrária que escolhemos para se anular na fronteira de  $\Omega$  e crescer suavemente até o valor unitário no interior do domínio. Como  $d = 0$  na fronteira,  $u = 1 - e^{\lambda x} \cos(2\pi y)$  ali por construção, a BC é satisfeita sem termo de penalidade.  $v$  e  $p$  são tratados de forma análoga, sendo  $v = \frac{\lambda}{2\pi} e^{\lambda x} \sin(2\pi y)$  e  $p = \frac{1}{2}(1 - e^{2\lambda x})$  as soluções analíticas correspondentes.

### Supervisionado vs. não-supervisionado

Antes do regime não-supervisionado pleno, rodamos uma sonda supervisionada: a rede recebe pares  $(x_i, y_i) \mapsto (u, v, p)$  amostrados da analítica, sem o termo da EDP. A loss é definida apenas pelos dados:

$$\mathcal{L}_{\text{sup}}(\theta) = \frac{1}{N} \sum_{n=1}^N (u - u^*)^2 + (v - v^*)^2 + (p - p^*)^2, \quad (11)$$

onde  $u^*, v^*, p^*$  são as parcelas analíticas de (10). É um teste de diagnóstico: se a rede não converge nem com a solução servida no prato, ela não converge buscando no escuro com o resíduo. Convergência supervisionada confirma que a arquitetura é *capaz* de expressar a solução; só então faz sentido testar se o resíduo é guia suficiente. Caso o supervisionado falhe, o caminho não é insistir no não-supervisionado, é mudar a arquitetura.

| Modo + arquitetura               | 1ª seed              | 2ª seed              | 3ª seed              | $\sigma$           |
|----------------------------------|----------------------|----------------------|----------------------|--------------------|
| Sup PINN ( $4 \times 32$ )       | $1,5 \times 10^{-3}$ | $1,7 \times 10^{-3}$ | $2,5 \times 10^{-3}$ | $5 \times 10^{-4}$ |
| Não-sup PINN ( $4 \times 32$ )   | $8,4 \times 10^{-4}$ | $1,4 \times 10^{-3}$ | $1,7 \times 10^{-3}$ | $4 \times 10^{-4}$ |
| Sup MixFunn ( $3 \times 1$ )     | $3,9 \times 10^{-2}$ | $2,4 \times 10^{-2}$ | $2,8 \times 10^{-2}$ | $8 \times 10^{-3}$ |
| Não-sup MixFunn ( $3 \times 1$ ) | $4,6 \times 10^{-2}$ | $2,0 \times 10^{-2}$ | $7,7 \times 10^{-2}$ | $3 \times 10^{-2}$ |

Tabela 1:  $L_{\text{val}}^2$  (loss) por seed dos modelos treinados contra a solução analítica (supervisionado) e contra o resíduo da EDP (não supervisionado), e  $\sigma$  (desvio padrão amostral). *Para reprodução: ver [Experimento 1 no GitHub](#).*

Os quatro grupos convergem na mesma ordem de magnitude ( $10^{-3}$  PINN,  $10^{-2}$  Mix). Por ser mais fácil, existe a possibilidade de o treino supervisionado atingir essa região em menos iterações,

mas todos os regimes acabam na mesma vizinhança do espaço de soluções. Por conta disso, a loss é equivalente. Já a variância entre seeds depende da natureza do problema e da estrutura da rede: as três primeiras linhas têm  $\sigma$  pequeno frente ao valor central, mas a MixFunn não supervisionada (quarta linha) varia substancialmente mais, com  $\sigma$  da mesma ordem do valor médio.

### Profundidade e largura

Estudamos como o tamanho da rede afeta a loss. Profundidade  $N$  (número de camadas) e largura  $n$  (número de neurônios por camada) são diretamente proporcionais ao tamanho da rede, e o tamanho controla sua expressividade.

| $N \backslash n$ | 16                   | 32                   | 64                                     |
|------------------|----------------------|----------------------|----------------------------------------|
| 4                | $2,0 \times 10^{-3}$ | $1,3 \times 10^{-3}$ | $1,9 \times 10^{-3}$                   |
| 6                | $2,1 \times 10^{-3}$ | $1,5 \times 10^{-3}$ | $1,2 \times 10^{-3}$                   |
| 8                | $2,2 \times 10^{-3}$ | $1,2 \times 10^{-3}$ | <b><math>1,0 \times 10^{-3}</math></b> |

PINN

| $N \backslash n$ | 1                    | 2                                      | 3                    |
|------------------|----------------------|----------------------------------------|----------------------|
| 1                | $7,1 \times 10^{-2}$ | $7,1 \times 10^{-2}$                   | $7,0 \times 10^{-2}$ |
| 2                | $3,1 \times 10^{-2}$ | <b><math>1,7 \times 10^{-2}</math></b> | $3,1 \times 10^{-2}$ |
| 3                | $2,3 \times 10^{-2}$ | $2,3 \times 10^{-2}$                   | $2,1 \times 10^{-2}$ |

MixFunn

Tabela 2:  $L^2_{\text{val}}$  (loss) por arquitetura para PINN (esquerda) e MixFunn (direita).  $N$ : número de camadas;  $n$ : número de neurônios por camada. Melhor caso em negrito. Para reprodução: ver [Experimento 1 no GitHub](#).

Assim como notamos na variância entre seeds (Tabela 1), a loss da maioria das arquiteturas testadas pela PINN é parecida: a rede tem mais parâmetros do que o necessário para esse problema, está hiperparametrizada, e ampliá-la a mantém na mesma região do espaço de soluções. É esperado que em detalhe a loss caia um pouco com o tamanho, vemos isso no  $8 \times 64$  ( $L^2 = 1,0 \times 10^{-3}$ , melhor da tabela). Mas pensando em eficiência,  $4 \times 16$  (915 params,  $L^2 = 2,0 \times 10^{-3}$ ) tem essencialmente a mesma loss que  $8 \times 64$  (29 507 params,  $L^2 = 1,0 \times 10^{-3}$ ), apesar de  $\sim 32 \times$  mais parâmetros.

Na MixFunn vemos uma barreira clara em  $7,1 \times 10^{-2}$  enquanto há apenas uma camada, independente do número de neurônios. A loss só desce a partir de duas camadas. A razão é estrutural: a solução analítica  $1 - e^{\lambda x} \cos(2\pi y)$  contém o produto de duas funções, e a MixFunn só gera produtos por composição de camadas, aumentar neurônios em uma única camada apenas soma funções (hiperparametrização imediata). Existe portanto uma limitação intrínseca: uma rede de uma camada não tem como representar produto, e a loss fica travada nesse patamar.

**MixFunn sof.** Para mostrar a barreira quantitativamente, treinamos uma variação também apresentada no paper da MixFunn:

$$\text{MixFunn}_{\text{sof}} = \sum_{k=1}^K \alpha_k f_k(z_k) + \sum_{i < j} \alpha_{ij} f_i(z_i) f_j(z_j), \quad (12)$$

onde *sof* é abreviação de *second-order function*, que habilita os produtos cruzados  $f_i \cdot f_j$  entre as funções atômicas dentro do mesmo neurônio. É distinto de empilhar uma segunda camada, mas também aumenta a expressividade da rede. Treinada como MixFunn sof  $1 \times 1$ , atinge  $L_{\text{val}}^2 = 1,0 \times 10^{-2}$ , abaixo do patamar de  $7,1 \times 10^{-2}$  no qual as três configurações de uma camada da Tabela 2 ficaram travadas: comprovamos que a barreira era de expressividade, não de capacidade.

Aqui o diagnóstico é fácil porque conhecemos a solução analítica e podemos isolar o termo crítico. Em problemas sem solução fechada, nunca sabemos a priori se chegamos numa barreira estrutural. Por isso é essencial variar a arquitetura para distinguir hiperparametrização de falta de expressividade. E por isso vale rodar a sonda supervisionada também. *Para reprodução: ver [Experimento 1 no GitHub](#).*

Uma das maneiras de analisar a capacidade de generalização da rede é explorar tanto resultados interpolados (dados de teste dentro da região de treino) quanto extrapolados (dados de teste fora da região de treino). Dentro da região de interpolação, a PINN ajusta a intensidade dos picos da função melhor que a MixFunn, o que justifica visualmente a loss menor da PINN observada na Tabela 2. Na extrapolação, ambas divergem progressivamente da analítica, como vemos em  $x = 2$ : pelo termo exponencial  $e^{\lambda x}$  com  $\lambda \approx -0,964$ , a analítica decai quase a zero, mas as redes só capturam esse decaimento dentro do intervalo de treino e depois voltam a subir com o sinal invertido.

Outro problema comum quando examinamos efeitos de extrapolação é a instabilidade numérica: mesmo que a rede generalize localmente, se a ordem de magnitude dos dados de entrada for muito diferente da do treino ela pode não conseguir reproduzir valores muito grandes ou muito pequenos, não é porque funcionou entre 0 e 1 que vai funcionar entre 0 e 100. Por exemplo, nas funções atômicas da MixFunn somamos constantes pequenas dentro de funções com singularidade na origem:  $\log(0,1 + \text{ReLU}(x))$  e  $\sqrt{0,01 + \text{ReLU}(x)}$ . Esses valores não são arbitrários: estão calibrados para a ordem de grandeza do nosso experimento. Um problema em outra escala provavelmente exigiria constantes diferentes.

Tradicionalmente, os gráficos de loss têm o formato em “taco de hóquei”: descida rápida nos primeiros mil passos e cada vez mais lenta depois. Por natureza do problema, a loss de treino é sempre decrescente. Os picos visíveis acontecem quando o passo do otimizador é maior que o vale local em que ele estava descendo, mas eventualmente o otimizador encontra o caminho de volta. Aumentar o tamanho do passo torna esses picos mais frequentes e maiores, e em excesso a loss pode não convergir, ajustar o passo é parte do cuidado experimental durante o treino.

A loss de treino, porém, não é igual à loss de teste: mais iterações não implicam loss de teste menor. Em geral ela satura em algum patamar, e pode até subir em cenários de *overfit*, quando a rede passa a aprender ruído dos dados em vez do padrão de fundo. Se estamos treinando uma rede com dados reais que têm ruído (típico do regime supervisionado), é preciso cuidado com hiperparametrização: a rede pode ajustar o ruído desses dados em vez do padrão físico, e a loss de teste piora à medida que a loss de treino continua a descer.

Pelo painel (a) da Figura 5 também notamos que o regime supervisionado não necessariamente

Kovasznyai --- campos  $u$  com BC Coons,  $x$  estendido até 2,5

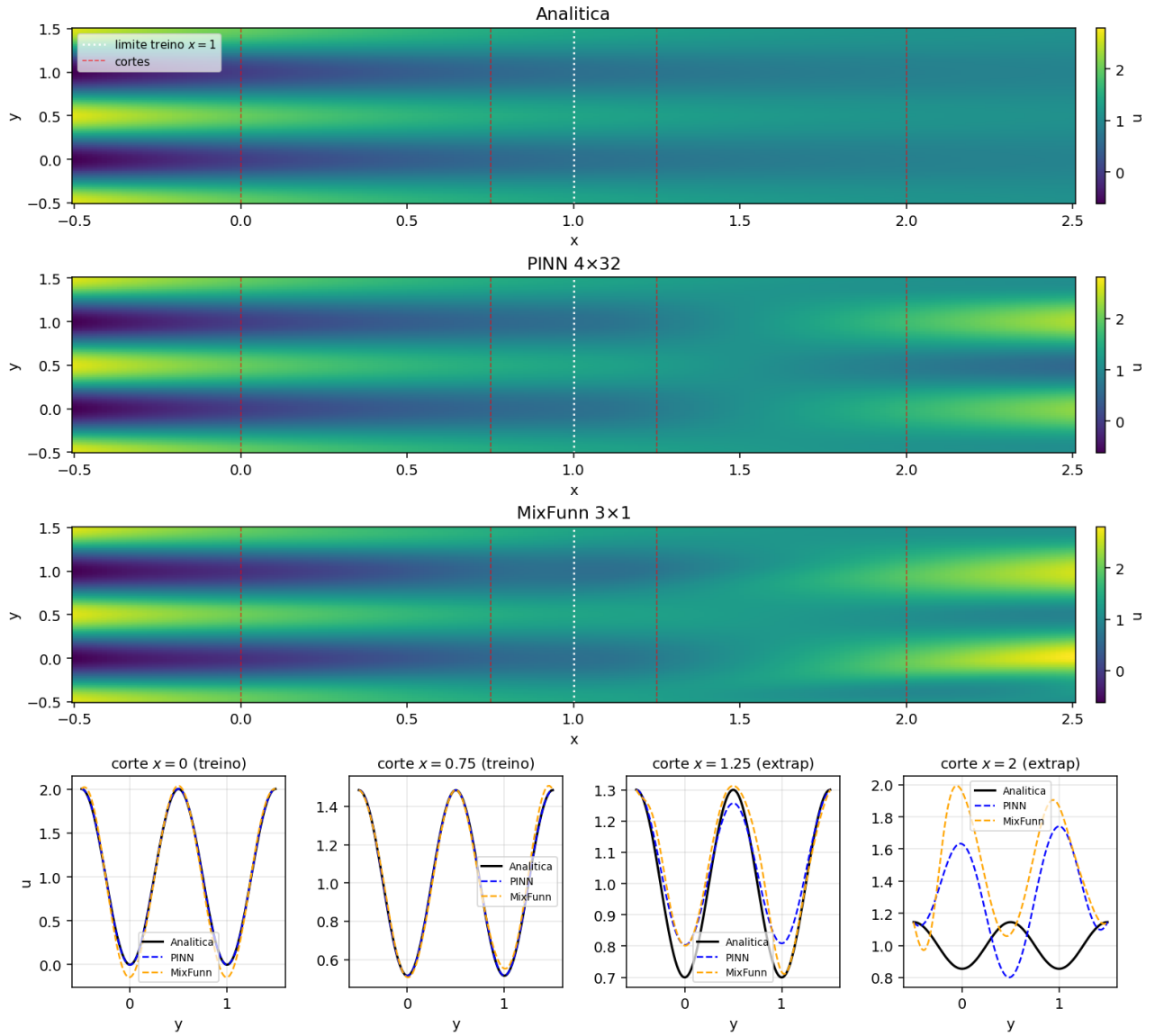


Figura 4: Análise do desempenho das duas redes da Tabela 2 dentro do domínio de treino,  $x \in [-0,5; 1]$  (interpolação), e fora do domínio de treino,  $x > 1$  (extrapolação). Para reprodução: ver [Experimento 1 no GitHub](#).

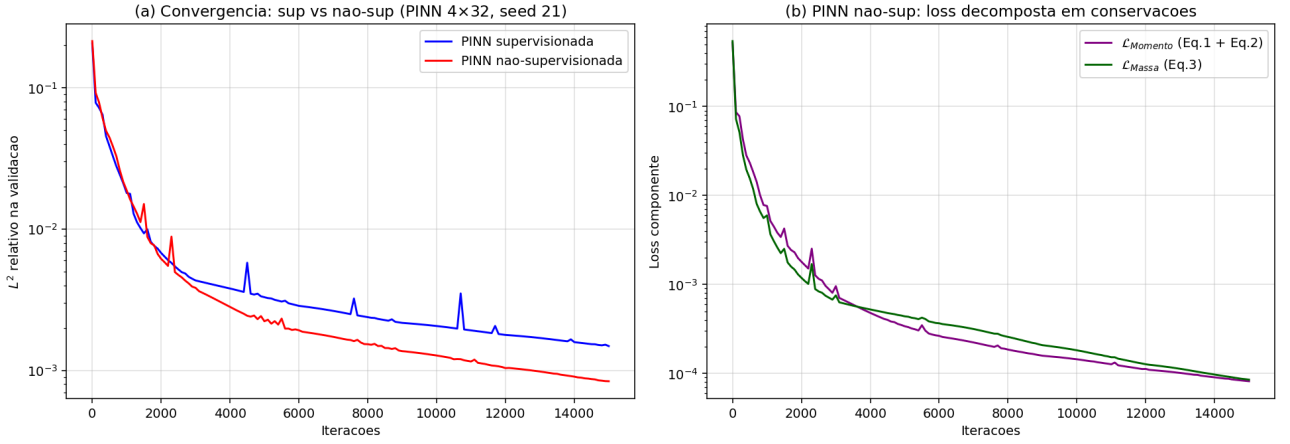


Figura 5: Loss em escala log. **(a)**  $L^2_{\text{val}}$  (loss) vs. iterações, PINN  $4 \times 32$  supervisionada (azul) e não supervisionada (vermelho), mesma seed. **(b)** Loss não supervisionada decomposta em  $\mathcal{L}_{\text{Momento}}$  (Eq.1+Eq.2) e  $\mathcal{L}_{\text{Massa}}$  (Eq.3). Para reprodução: ver [Experimento 1 no GitHub](#).

aprende mais rápido, apesar de ser um treino “mais fácil”. Pelo painel (b), cada componente da loss de resíduo tem influência diferente no aprendizado. Uma tática comum para colocar um viés no loop de treino é multiplicar cada componente por um peso  $\lambda$ ,  $\mathcal{L} = \lambda_{\text{Mom}} \mathcal{L}_{\text{Mom}} + \lambda_{\text{Mas}} \mathcal{L}_{\text{Mas}}$ , por exemplo dando mais peso à conservação de momento do que à de massa quando a natureza do sistema indica que aquela componente é mais crítica. Os  $\lambda$  não precisam ser constantes: podem variar ao longo do treino, aprendendo primeiro uma componente e depois outra. Esse é mais um aspecto do caráter experimental dessas técnicas. Cada problema exige exploração própria do espaço de soluções e dos hiperparâmetros: é mais parecido com estarmos num laboratório explorando um problema empírico do que resolvendo um código determinístico.

## Regularização

Também testamos dois métodos muito comuns de regularização, *dropout* e *sub-amostragem*, em cada uma das duas redes.

| Configuração               | PINN ( $4 \times 32$ ) | MixFunn ( $3 \times 1$ ) |
|----------------------------|------------------------|--------------------------|
| baseline (sem)             | $1,1 \times 10^{-3}$   | $4,6 \times 10^{-2}$     |
| dropout 10%                | $6,2 \times 10^{-2}$   | $8,3 \times 10^{-2}$     |
| dropout 15%                | $9,0 \times 10^{-2}$   | $3,0 \times 10^{-2}$     |
| sub-amostra 70%            | $1,0 \times 10^{-3}$   | $2,1 \times 10^{-2}$     |
| sub-amostra 50%            | $1,2 \times 10^{-3}$   | $2,1 \times 10^{-2}$     |
| combo (drop 10% + sub 50%) | $7,5 \times 10^{-2}$   | $2,5 \times 10^{-2}$     |

Tabela 3:  $L^2_{\text{val}}$  (loss) variando dois métodos de regularização (*dropout* e *sub-amostragem*) para a PINN  $4 \times 32$  e a MixFunn  $3 \times 1$ . Para reprodução: ver [Experimento 2 no GitHub](#).

A sub-amostragem não fez diferença na PINN, mas na MixFunn reduziu a loss pela metade; já o

dropout piorou a loss em todos os casos, com a única exceção do dropout de 15% na MixFunn. Esse comportamento tem caráter apenas qualitativo: uma caracterização rigorosa exigiria variância de *seed* ampla e maior granularidade nas frações de dropout e sub-amostragem. Ainda assim, reforça o caráter empírico da exploração, em que todos os parâmetros importam.

## Pruning e Annealing

Um dos pontos fortes da MixFunn é sua representatividade física: o design facilita a extração das equações analíticas que a rede aprende.

Todo o treino da MixFunn neste documento usa annealing da temperatura  $T$  da softmax:  $T$  desce linearmente de 5,0 a 0,05 ao longo do treino, afiando a softmax e concentrando peso nas poucas funções da base que de fato participam da solução. Como as funções atômicas da MixFunn já são muito expressivas, a rede tende à hiperparametrização; o annealing força a escolha de poucos  $\alpha$ 's relevantes.

Aplicamos pruning às duas arquiteturas de naturezas diferentes: MixFunn  $3 \times 1$  e MixFunn<sub>sof</sub>  $1 \times 1$ , vinda da Equação (12).

| Ratio $r$             | $\alpha$ zerados / total | $L_{\text{val}}^2$   | Ratio $r$                           | $\alpha$ zerados / total | $L_{\text{val}}^2$   |
|-----------------------|--------------------------|----------------------|-------------------------------------|--------------------------|----------------------|
| 0 ( <i>baseline</i> ) | 0 / 35                   | $9,8 \times 10^{-2}$ | 0 ( <i>baseline</i> )               | 0 / 105                  | $4,4 \times 10^{-3}$ |
| 0,30                  | 10 / 35                  | $1,3 \times 10^{-1}$ | 0,30                                | 31 / 105                 | $4,4 \times 10^{-3}$ |
| 0,50                  | 16 / 35                  | $2,5 \times 10^{-1}$ | 0,50                                | 52 / 105                 | $4,4 \times 10^{-3}$ |
| 0,70                  | 22 / 35                  | $5,7 \times 10^{-1}$ | 0,70                                | 73 / 105                 | $9,1 \times 10^{-2}$ |
| 0,90                  | 30 / 35                  | $5,6 \times 10^{-1}$ | 0,90                                | 94 / 105                 | $2,3 \times 10^{-1}$ |
| MixFunn $3 \times 1$  |                          |                      | MixFunn <sub>sof</sub> $1 \times 1$ |                          |                      |

Tabela 4: Loss após pruning da MixFunn  $3 \times 1$  e MixFunn<sub>sof</sub>  $1 \times 1$  para diferentes ratios. *Para reprodução: ver [Experimento 3 no GitHub](#).*

Na MixFunn  $3 \times 1$ , cerca de 30% dos parâmetros mostram-se inúteis; a partir daí a loss começa a mudar significativamente. Já a MixFunn<sub>sof</sub>  $1 \times 1$  tolera o pruning de até metade dos parâmetros sem alterar a loss, sinal de que está hiperparametrizada — o que já é a intenção da MixFunn, e com o *sof* é esperado que boa parte dos parâmetros fique inútil. Mesmo com o pruning ratio de 30%, ainda restam muitos parâmetros significantes, então não conseguimos escrever uma equação analítica para comparar com  $u = 1 - e^{\lambda x} \cos(2\pi y)$ ; e para a MixFunn<sub>sof</sub>  $1 \times 1$ , mesmo com o pruning ratio de 70% a loss permanece pequena frente ao colapso da  $3 \times 1$ , mas o número de parâmetros significantes ainda impede extrair essa forma fechada. Um annealing mais agressivo ou escolher menos funções atômicas poderiam ajudar esse processo, mas é preciso tomar cuidado para não cair em *lookahead bias*, que é definir uma arquitetura utilizando conhecimento prévio da solução.



### 3.2 Aneurisma intracraniano 3D

Após Kovasznay (§3.1), passamos a uma aplicação real para mostrar o desempenho das redes em geometria complexa. Inspiramo-nos no *Hidden Fluid Mechanics* (HFM) [Raissi 2018], que infere velocidade e pressão num segmento cerebrovascular a partir de medidas de um escalar passivo (corante) injetado no escoamento. Aqui, baixamos os dados do *dataset* ANEUMO [Li 2025]: uma nuvem de pontos de aproximadamente  $8 \text{ mm}^3$  extraída de um paciente real, representando um trecho da árvore vascular cerebral com um aneurisma — um tubo com uma bolsa lateral, o saco aneurismático. Esse tipo de simulação serve, por exemplo, para estimar a pressão dentro do saco e prever risco de ruptura.

Em cada ponto da nuvem, o *dataset* já traz a solução pré-computada pelo CFD comercial Fluent: vetor de velocidade  $\mathbf{u} = (u, v, w)$  em m/s e pressão  $p$  em Pa, já que o cálculo numérico direto desses campos é complexo. O problema é definido pelas equações de Navier–Stokes incompressíveis, com velocidade zero na parede (definida pela nuvem de pontos do aneurisma), perfil de velocidade fisiológico  $\mathbf{u}_{\text{fis}}$  na entrada (extraído do *dataset*) e pressão zero na saída:

$$\begin{cases} (\mathbf{u} \cdot \nabla)\mathbf{u} + \nabla p / \rho - \nu \nabla^2 \mathbf{u} = 0, \\ \nabla \cdot \mathbf{u} = 0, \\ \mathbf{u}|_{\text{parede}} = 0, \quad \mathbf{u}|_{\text{entrada}} = \mathbf{u}_{\text{fis}}, \quad p|_{\text{saída}} = 0. \end{cases} \quad (13)$$

Fizemos um estudo variando o quanto os dados ANEUMO entravam no treino, e a rede só conseguiu aprender no regime 100% supervisionado.

| Configuração                                         | Supervisionado | MSE                                     |
|------------------------------------------------------|----------------|-----------------------------------------|
| PINN $5 \times 64$                                   | 0%             | $1,80 \times 10^{-2}$                   |
| <b>PINN <math>8 \times 128</math></b>                | <b>0%</b>      | <b><math>1,78 \times 10^{-2}</math></b> |
| MixFunn $3 \times 1$                                 | 0%             | $1,92 \times 10^{-2}$                   |
| MixFunn <sub>sof</sub> $3 \times 1$                  | 0%             | $1,83 \times 10^{-2}$                   |
| PINN $5 \times 64$                                   | 25%            | $1,32 \times 10^{-2}$                   |
| PINN $5 \times 64$                                   | 50%            | $1,33 \times 10^{-2}$                   |
| MixFunn <sub>sof</sub> $2 \times 2$                  | 50%            | $1,38 \times 10^{-2}$                   |
| <b>PINN <math>5 \times 64</math></b>                 | <b>100%</b>    | <b><math>1,81 \times 10^{-3}</math></b> |
| <b>MixFunn<sub>sof</sub> <math>2 \times 2</math></b> | <b>100%</b>    | <b><math>3,76 \times 10^{-3}</math></b> |

Tabela 5: Variações de PINN e MixFunn treinadas contra o *dataset* ANEUMO. Linhas em negrito indicam as configurações utilizadas nas Figuras 6 e 7. Para reprodução: ver [Experimento 4 no GitHub](#).

Isso mostra que as arquiteturas escolhidas *são capazes* de representar a solução: quando guiadas pelos dados, ambas encontram uma solução; o que ainda precisamos melhorar é o algoritmo de busca por resíduo da EDP. Vale notar que o HFM também não opera apenas com o resíduo da EDP — eles usam observações de um escalar passivo (corante) como dado adicional, e mesmo assim o treino demanda semanas em GPU dedicada. Uma alternativa complementar, que propomos como direção de trabalho futuro, seria aprendizado em etapas: treinar a rede primeiro em regimes de Reynolds mais

baixo (por exemplo,  $Re = 10$ ), herdar os pesos resultantes como inicialização do treino em regimes mais altos ( $Re = 100$ ,  $Re = 300$ ) e progredir gradualmente até  $Re \sim 700$ , deixando a rede resolver problemas mais simples antes do mais complexo. Métodos de normalização dos campos físicos ou da loss também podem ajudar. As Figuras 6 e 7 mostram o *ground truth* ANEUMO e três configurações treinadas: PINN supervisionada, MixFunn<sub>sof</sub> supervisionada e PINN  $8 \times 128$  não-supervisionada. Visualmente, as duas redes supervisionadas recuperam a estrutura geral do escoamento (com perda de detalhe na pressão), enquanto a não-supervisionada não aprendeu o problema.

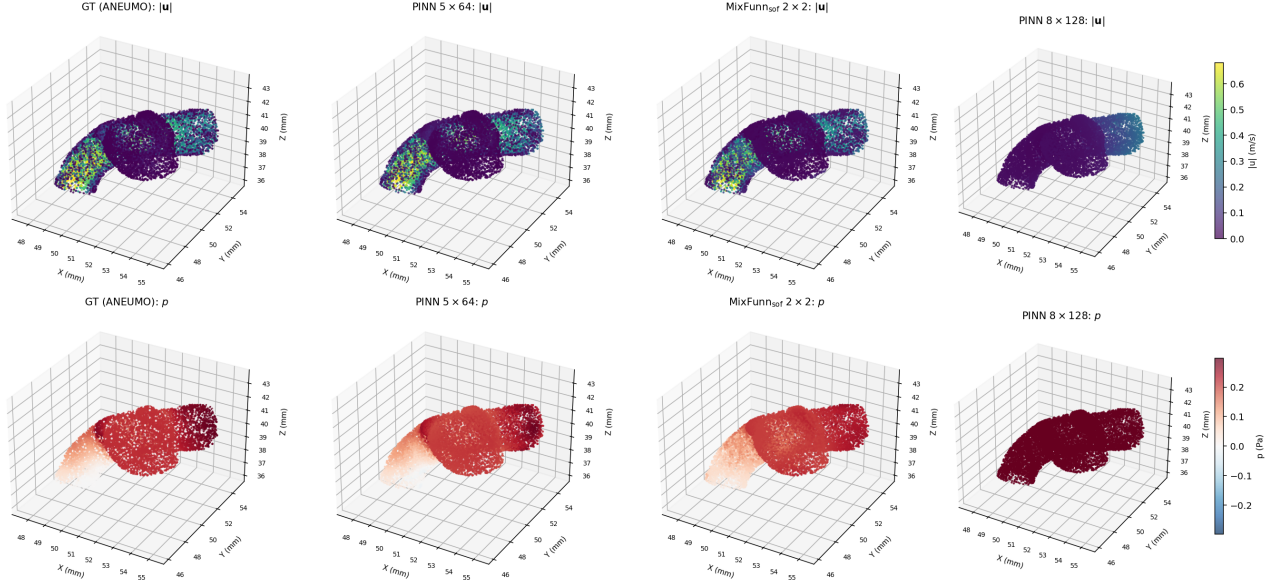


Figura 6: Nuvem 3D colorida por  $|u|$  (linha superior) e  $p$  (linha inferior). Colunas, em ordem crescente de loss: *ground truth* ANEUMO, PINN supervisionada, MixFunn<sub>sof</sub> supervisionada, PINN  $8 \times 128$  não-supervisionada. Para reprodução: ver [Experimento 4 no GitHub](#).

### 3.3 Diversidade de EDPs

Os próximos três problemas são tratados em visão geral qualitativa, demonstrando a aplicabilidade do par PINN e MixFunn em regimes físicos distintos. **Todos os resultados a seguir são treinados em regime não-supervisionado**, pelo resíduo da EDP mais condições de contorno, sem usar a referência numérica durante o treino.

#### Equação de Burgers viscosa

A equação de Burgers viscosa é um modelo de advecção-difusão não-linear em mecânica dos fluidos: a condição inicial  $-\sin(\pi x)$  evolui para um gradiente acentuado próximo a  $t = 1$ , análogo a um choque amortecido pela viscosidade. Como mostra a Figura 8, tanto a PINN quanto a MixFunn<sub>sof</sub> recuperam o perfil de referência, incluindo a região de gradiente abrupto.

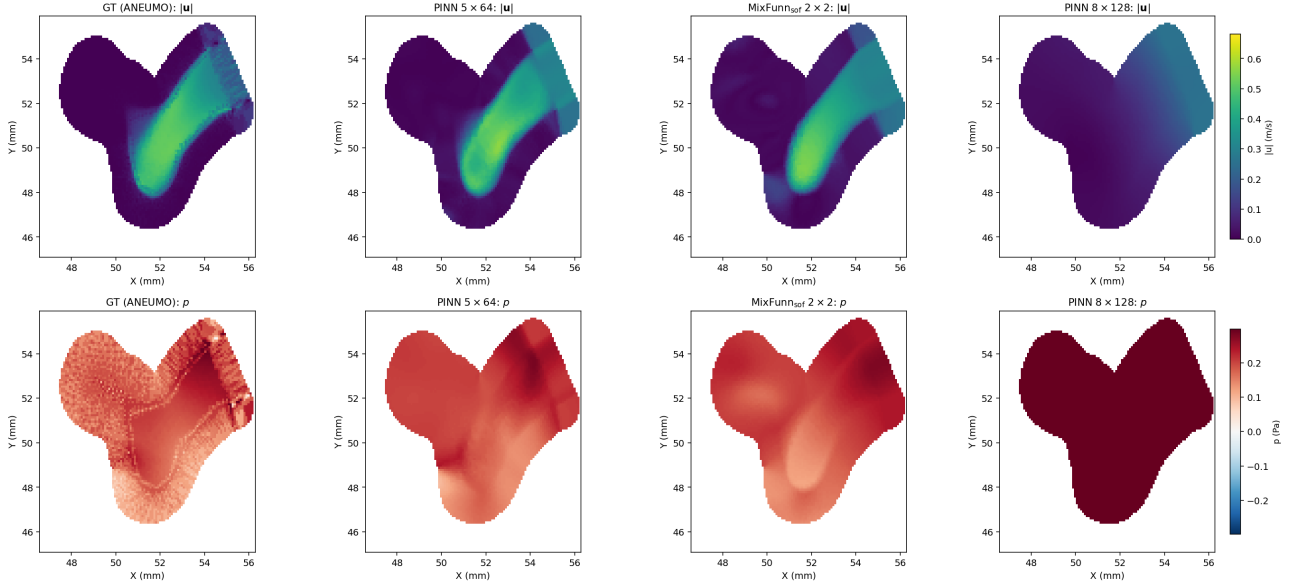


Figura 7: Mesmos campos da Fig. 6 no plano central  $z \approx 39,75$  mm.

$$\begin{cases} u_t + u u_x = \nu u_{xx}, \\ u(x, 0) = -\sin(\pi x), & \nu = 0,01/\pi, \quad x \in [-1, 1], \quad t \in [0, 1]. \\ u(\pm 1, t) = 0, \end{cases} \quad (14)$$

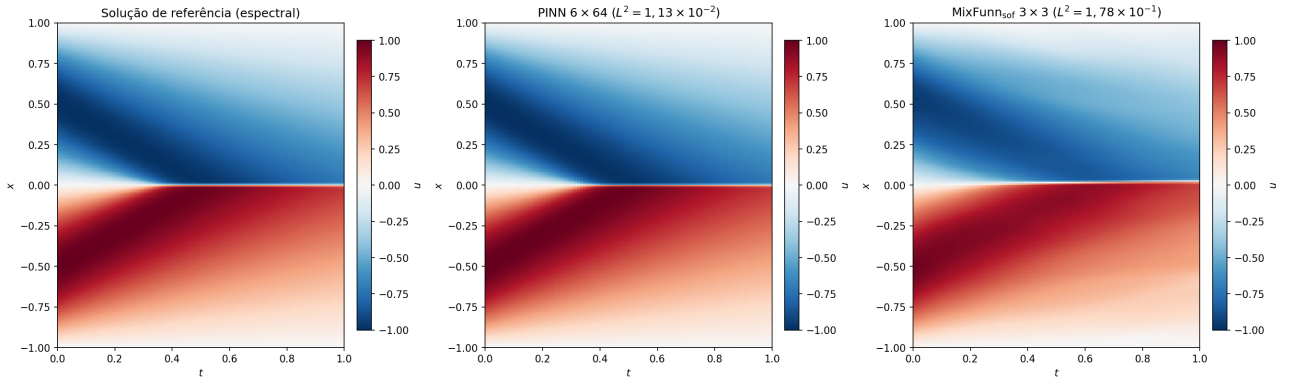


Figura 8: Burgers viscosa não-supervisionada, PINN e MixFunn<sub>sof</sub> comparados à solução de referência [Raissi 2019]. Para reprodução: ver [Experimento 5 no GitHub](#).

### Equação de Schrödinger não-linear

A equação de Schrödinger não-linear modela pacotes de onda com auto-interação, comum em óptica não-linear e condensados de Bose–Einstein; a condição inicial  $\psi(x, 0) = 2 \operatorname{sech}(x)$  define um sóliton que propaga seu envelope. Como mostra a Figura 9, a PINN reproduz o sóliton fielmente, enquanto a MixFunn<sub>sof</sub> subestima a amplitude do pico central mas recupera bem o decaimento nas extremidades.

$$\begin{cases} i\psi_t + \frac{1}{2}\psi_{xx} + |\psi|^2\psi = 0, \\ \psi(x, 0) = 2 \operatorname{sech}(x), \\ \psi(\pm 5, t) = 0, \end{cases} \quad t \in [0, \pi/2]. \quad (15)$$

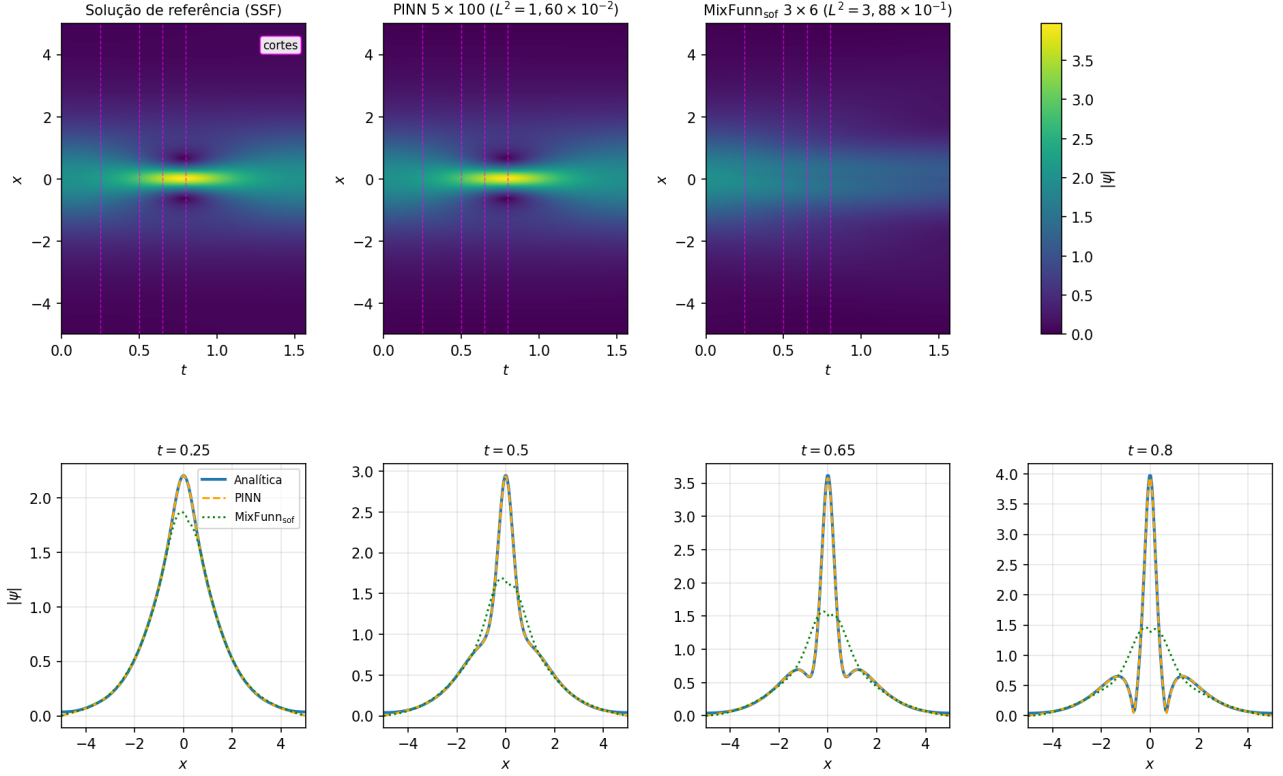


Figura 9: Schrödinger não-linear não-supervisionada, PINN e MixFunn<sub>sof</sub> comparados à solução de referência [Raissi 2019]. Para reprodução: ver [Experimento 6 no GitHub](#).

### Espalhamento eletromagnético com disco dielétrico

O problema é magnetostático 2D: um disco de permeabilidade  $\mu_r = 3$  em um campo uniforme deforma as linhas de fluxo no entorno, exigindo que a rede satisfaça as equações de Maxwell sem fontes inclusive através da descontinuidade na borda do disco. Como mostra a Figura 10, a PINN recupera o padrão perturbado com fidelidade, enquanto a MixFunn<sub>sof</sub> atinge loss superior, mas reproduz bem a estrutura na vizinhança do disco.

$$\begin{cases} \nabla \times \mathbf{H} = 0, \\ \nabla \cdot (\mu \mathbf{H}) = 0, \\ \mathbf{H}|_{\partial\Omega} = (0, 0, 1), \end{cases} \quad \mu_r = \begin{cases} 3, & \|(x, z) - (0, 5, 0, 5)\| < 0,2 \\ 1, & \text{caso contrário} \end{cases} \quad (16)$$

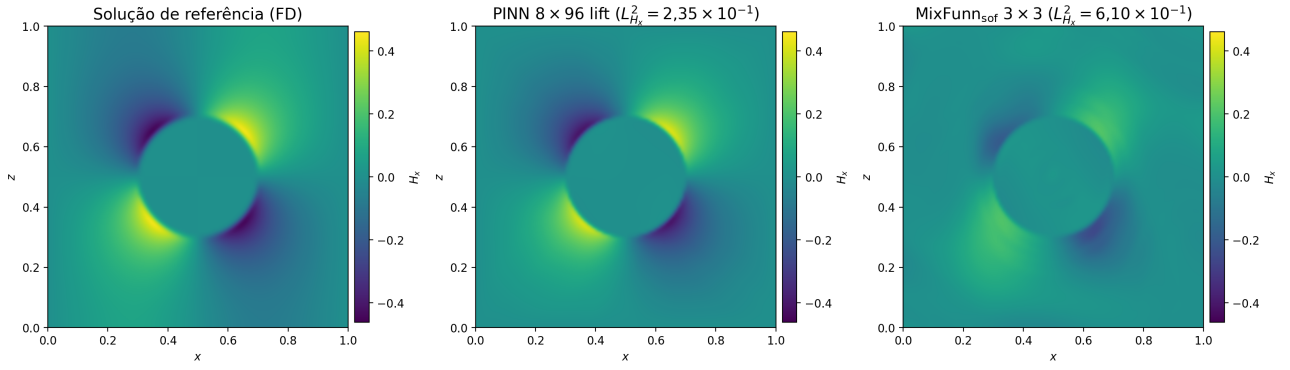


Figura 10: Magnetostática 2D não-supervisionada, componente  $H_x$ : PINN e MixFunn<sub>sof</sub> comparados à solução de referência via diferenças finitas (problema de [Nohra 2024]). *Para reprodução: ver [Experimento 7 no GitHub](#).*

## 4 CONCLUSÕES E CONSIDERAÇÕES FINAIS

Os experimentos demonstraram que tanto a PINN quanto a MixFunn são capazes de representar soluções de equações diferenciais parciais em uma diversidade de regimes físicos, validando o paradigma de redes informadas por física como uma alternativa viável aos métodos numéricos tradicionais. A escolha entre as duas arquiteturas depende do perfil do problema: a PINN se mostra mais robusta em escoamentos complexos, onde a expressividade ampla compensa o número maior de parâmetros, enquanto a MixFunn brilha em problemas com estrutura simbólica simples, oferecendo soluções mais compactas e interpretáveis. O caráter empírico observado ao longo do trabalho — arquitetura, otimizador, tratamento das condições de contorno e métodos de regularização precisando ser explorados caso a caso — sugere que esse tipo de pesquisa lembra mais o trabalho de laboratório do que a execução determinística de um método numérico clássico.

Como sugestões para trabalhos futuros, destacam-se: (i) um estudo quantitativo das análises feitas neste trabalho, uma vez que toda a validação aqui apresentada teve caráter qualitativo — na maioria dos experimentos não foi feita variância de *seed*, e os resultados têm o papel de ilustrar conceitos, não de estabelecer parâmetros estatisticamente robustos, o que complementaria a validação inicial; (ii) aplicação a outras equações diferenciais parciais em regimes físicos ainda não explorados; (iii) estudo do aneurisma intracraniano tridimensional em regime não-supervisionado, com técnicas como aprendizado em etapas em número de Reynolds crescente, métodos de normalização específicos, e variação sistemática de hiperparâmetros, todas visando melhorar o algoritmo de busca da solução pelo resíduo da EDP; (iv) exploração de outras arquiteturas de redes informadas por física, e de variantes híbridas combinando PINN e MixFunn.

## REFERÊNCIAS

- 1 RAISSI, Maziar; PERDIKARIS, Paris; KARNIADAKIS, George Em. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019. Disponível em: <https://doi.org/10.1016/j.jcp.2018.10.045>.
- 2 FARIAS, Tiago de Souza; LIMA, Gubio Gomes de; MAZIERO, Jonas; VILLAS-BOAS, Celso Jorge. MixFunn: a neural network for differential equations with improved generalization and interpretability. **arXiv preprint arXiv:2503.22528**, 2025. Disponível em: <https://arxiv.org/abs/2503.22528>.
- 3 GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron. **Deep Learning**. Cambridge: MIT Press, 2016. Disponível em: <https://www.deeplearningbook.org>.
- 4 RAISSI, Maziar; YAZDANI, Alireza; KARNIADAKIS, George Em. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. **Science**, v. 367, n. 6481, p. 1026–1030, 2020. Disponível em: <https://arxiv.org/abs/1808.04327>.
- 5 LI, Yuhe et al. ANEUMO: A high-quality dataset for aneurysm-related cerebral hemodynamics. **arXiv preprint arXiv:2505.14717**, 2025. Disponível em: <https://arxiv.org/abs/2505.14717>.
- 6 NOHRA, Michel; DUFOUR, Steven. Physics-Informed Neural Networks for the Numerical Modeling of Steady-State and Transient Electromagnetic Problems with Discontinuous Media. **arXiv preprint arXiv:2406.04380**, 2024. Disponível em: <https://arxiv.org/abs/2406.04380>.
- 7 MIRANDA, Gustavo Cafe de; LIMA, Gubio G. de; FARIAS, Tiago de S. An introduction to neural networks for physicists. **arXiv preprint arXiv:2505.13042**, 2025. Disponível em: <https://arxiv.org/abs/2505.13042>.

## CÓDIGO

Todos os *scripts* de pré-processamento, treino, validação e visualização estão no repositório <https://github.com/NicolasRossoni/NNinPhysics>. O *dataset* ANEUMO está disponível em <https://arxiv.org/abs/2505.14717>.